Web Scraping Report — Task 10

Novexa Technologies · Data Analysis Internship

1. Objective

Collect structured data from a publicly accessible website using Python, clean it, and store it in CSV format, using **requests** and **BeautifulSoup**.

2. Target Site

https://books.toscrape.com/ — a fictional bookstore built and maintained specifically as a web-scraping sandbox (see toscrape.com). It has no login wall, no rate limiting, and explicitly invites scraping, so it satisfies the task's requirement to respect Terms of Service and robots.txt.

3. Tools & Libraries

- **requests** — HTTP GET requests to the catalogue and product pages
- **BeautifulSoup (bs4)** — HTML parsing
- **pandas** — data storage, cleaning, CSV export
- Python's built-in **re** — regex cleanup of price/availability text

4. Methodology

- **Discover pagination** — the catalogue is paginated at /catalogue/page-N.html, 20 books per page, 50 pages total (1000 books).
- **Listing page parsing** — each article.product_pod element on a listing page yields a link to the book's own product page.
- **Product page parsing** — extract Title, Price, Rating (decoded from a CSS class like star-rating Three), Availability + stock count, Category (breadcrumb), UPC, cover image URL, and product page URL.
- **Cleaning** — currency symbols stripped, ratings converted from words to integers, availability converted to boolean + numeric stock count, duplicate URLs dropped.
- **Storage** — all records loaded into a pandas DataFrame and exported to data/books_dataset.csv.
- **Politeness** — a 0.2s delay between requests and a descriptive User-Agent string.

5. Sample Findings

The repository ships a 60-book sample (catalogue pages 1–3), collected live from the site during development:

Metric	Value
Books collected (sample)	60
Average price	£35.00
Cheapest book	£12.84 — In Her Wake
Most expensive book	£57.31 — Slow States of Collapse: Poems
Books in stock	60 / 60 (100%)

Running **scrape_books.py --pages 50** reproduces this at full scale (~1000 books) with every field, including star rating, stock count, category and UPC.

6. Challenges Encountered

- **Star ratings are not plain text** — encoded as a CSS class (e.g. star-rating Three), so the scraper reads the class list and maps the word to an integer.
- **Long titles get truncated on listing pages** — the full title is only reliable on the product's own page, so the script scrapes there for that field.
- **No fixed page total in the HTML** beyond a 'Page X of 50' footer, so the script requests page-N.html until it hits a 404 — robust to the catalogue changing size.
- **Politeness vs. speed** — 1000 products means 1000+ requests; a small per-request delay avoids overloading the sandbox server, at the cost of a longer runtime (~5–8 minutes for the full site).

7. Interview Questions — Short Answers

What is web scraping?
Automatically extracting data from websites by fetching their HTML and parsing out the pieces of information you need, instead of copying it by hand.

What is BeautifulSoup?
A Python library for parsing HTML/XML documents into a navigable tree, so you can search for tags, classes, and attributes to pull out specific content.

What is the Requests library used for?
Sending HTTP requests (GET, POST, etc.) from Python and receiving the response — in scraping, it downloads the page's HTML.

What is HTML parsing?
Converting raw HTML text into a structured tree of elements/tags so a program can query and extract specific content instead of treating the page as one long string.

What is the purpose of HTTP requests?
They're the mechanism by which a client asks a server for a resource — a web page, an image, an API response — and receives it back.

What is the difference between static and dynamic websites?
A static site's HTML is fully generated server-side and can be read directly with requests + BeautifulSoup. A dynamic site builds/updates content in the browser via JavaScript after the initial load, so scraping it typically needs browser automation (e.g. Selenium, Playwright) instead.

8. Conclusion

The scraper successfully collects structured book data (title, price, rating, availability, category, and more) from a live website, cleans it, and exports it to CSV — demonstrating the full web scraping pipeline from HTTP request through to an analysis-ready dataset.